

Chaos Dungeon

Tác giả: Đoàn Minh Đức

Contents

- **Overview**
- **Game Design**
 - Story
 - Control
 - Gameplay
 - Architecture
- **Implementation**
 - Roles / Assignments
 - Assets / Tools / Tutorials
 - Techniques
 - Working history / Problems & Solutions

Contents

- **Summary**
 - Pros & Cons
 - Lesson learned
 - Future development
- **Demo**

Overview

- Game Title: Chaos Dungeon
- Genre: 2D Roguelike / Action Dungeon Crawler
- Platform: PC (Unity Engine)
- Target Audience: Players who enjoy fast-paced combat and procedural dungeon exploration

Game Design - Story

- The player enters a mysterious dungeon filled with chaos energy.
- Each run is different due to procedural generation.
- The goal is to survive and uncover deeper levels of the dungeon.

Game Design - Control

- Keyboard
 - Movement: WASD
 - Attack: Left Mouse Click
- Mouse
 - Aim / Direction

KeyBoard	Function
WASD	Movement
B	Open Store
M	Open Big Map
ESC	Pause / Menu



Gameplay

- **Characters**

- Player character with stats:
 - Health
 - Damage
 - Speed

- **Enemies**

- Different enemy types with AI behaviors
- Increasing difficulty over levels map of dungeon



- **Rules**

- Adaptive Dungeon Generation Rules

- The dungeon is dynamically generated based on the player's performance in the previous run.
 - The system analyzes player metrics, including:
 - Equipment (gear quality, upgrades)
 - Remaining health
 - Completion time
 - Based on these parameters, the game adjusts:
 - Dungeon size (numbers of rooms)
 - Room complexity
 - Enemy density
 - Players with stronger performance will face larger and more challenging dungeons, while weaker players will receive smaller and more manageable maps



- **Rules**

- **Enemy Scaling Rules**

- Enemy strength is influenced by the player's previous performance.
 - Adjustments include:
 - Enemy health
 - Enemy speed
 - Damage output
 - Spawn rate
 - This ensures a balanced difficulty curve and maintains challenge without overwhelming the player



- **Escape Mechanic Rules**

- To escape the dungeon, the player must:
 - Collect all required keys
 - Locate the correct exit door
 - Multiple doors may exit, but only one is the real exit



- **Maps & Missions**
 - Procedural ly generated dungeon
 - Rooms connected randomly
- **Weapons/Skills/Items**
 - Skills upgrade system
- **Shop**
 - Buy upgrades using gold
 - Improve survivability and damage



Implementation – Assignments

Personal project (independent development)

Roles	Member
Leader	Đoàn Minh Đức
Artist	Đoàn Minh Đức
Coder	Đoàn Minh Đức
Tester	Đoàn Minh Đức

Implementation - Assets

- **Nhân vật:** pixel (Youtube, assets Unity store)
- **Đồ họa:** tile 2D
- **Vũ khí:** Prefab từ assets Unity(Gun, Sword).
- **Maps:** design with assets map Unity.

Implementation - Tools

- Engine: Unity
- IDE: Visual Studio 2022
- Tweening: DOTween
- Version Control: Git
- Asset: (Free assets)

Implementation - Tutorials

- Youtube
- [Unity Docs](#)
- Google
- Chat AI GPT, gemini,...

Implementation - Techniques

- **Optimization Techniques**

- Object pooling: Used to reuse game objects (enemies, bullets, effect, coin, item,...) instead of instantiating and destroying them repeatedly, improving performance and reducing garbage collection.

- **Data Management**

- ScriptableObject:
 - Used to store and manage game data such as.
 - Enemy
 - Item

Implementation - Techniques

- **Procedural Content Generation**

- Procedural Generation System: Generates dungeon layouts dynamically to ensure replayability.
- Binary Space Partitioning (BSP): Used to divide the dungeon into rooms and create a structured layout.
- Breadth-First Search (BFS): This application is used to find the way to the real door, ensuring that the player can reach the destination door.
- Perlin Noise: Used to create natural variations in obstacle placement and environment details.

Implementation - Techniques

- **Design Patterns**

- Singleton Pattern: Ensures a single instance for core managers (GameManager, UIManager, AudioManager, ...).
- Observer Pattern: Handles event-driven communication between systems (UI updates, player / enemy stats changes, pooling).

- **UI System**

- Canvas Group: Controls UI transitions (fade in/out, interactability, visibility).

Implementation - Techniques

- **UI System**

- Canvas Group: Controls UI transitions (fade in/out, interactability, visibility).

- **AI & Navigation**

- NavMesh AI: Used for enemy pathfinding and movement behavior.

- **Animation System**

- Unity Animation System: Controls character animations, state transitions, and combat feedback.

FUTURE DEVELOPMENT

- Adding more features when creating a new map
- Add character
- Add weapons and skills
- Add cutscene functionality
- Add other enemies: BOSS

Working history / Problems & Solutions

- **Progress:**

- Completed random map creation
- Saved data when players win
- Weapon and worm shop
- vvv

- **Problems:**

- Map creation time is still too long
- Warnings in the map are not being analyzed correctly
- Item creation is not aesthetically pleasing
- Few characters
- vvv